

Documento de Referencia: **Plan de Proyecto.**

Proyecto: Aplicación: **“No Te Cortes”**

Sector: **Servicios de Estética e Imagen Personal (CNAE 9602)**

1. Objetivos y Alcance

Objetivos

- **Objetivo Principal:** Desarrollar una aplicación móvil nativa (Android) para la gestión automatizada de citas y servicios de la barbería/peluquería "NoTeCortes".
- **Objetivos Específicos:**
 1. Digitalizar la agenda para permitir reservas 24/7 sin intervención manual.
 2. Implementar un sistema de notificaciones push vía FCM para confirmar citas y reducir el absentismo en un 20%.
 3. Gestionar tres niveles de acceso (Admin, Empleado, Cliente) con permisos diferenciados.
 4. Facilitar el control de personal, bloqueo de fechas y gestión de servicios en tiempo real.

Alcance Funcional

- Gestión de Citas: Reserva en 3 pasos para clientes y gestión manual para el personal.
- Sistema de Notificaciones: Envío automático de confirmaciones y alertas de marketing.
- Panel Administrativo: CRUD de servicios, precios, promociones y gestión de plantilla.
- Seguridad: Validación de usuarios por rol, estados activos/inactivos y recuperación de credenciales.

Exclusiones (Fuera de Alcance)

- Pagos In-App: El cobro se realizará presencialmente en el local (Efectivo/TPV).
- Venta de Productos: No se incluye módulo de e-commerce.
- Soporte iOS: Proyecto exclusivo para el ecosistema Android.
- Migración de Datos: Se parte de una base de datos limpia; no se importan agendas antiguas.

2. Ciclo de Vida: Modelo Incremental

Se ha seleccionado el **Modelo de Ciclo de Vida Incremental** para el desarrollo de "No Te Cortes". Este enfoque permite construir la aplicación en bloques funcionales que se agregan a un núcleo común, permitiendo que el sistema crezca de forma operativa y segura.

2.1. Motivación de la elección

La principal motivación es la **jerarquía de dependencias** del proyecto. En una aplicación de servicios, no se puede "reservar" si no hay "servicios" creados, y no se pueden "crear servicios" si el usuario no tiene "permisos de administrador".

El modelo incremental nos permite atacar este orden lógico:

1. **Núcleo:** Base de Datos y Seguridad (Indispensable).
2. **Incremento 1:** Gestión Administrativa (Carga de datos reales).
3. **Incremento 2:** Interfaz de Cliente (Consumo de esos datos).

La elección del **Modelo Incremental** no solo responde a la jerarquía funcional de la aplicación, sino que es técnicamente imperativa al trabajar con servicios **Cloud Native como Firebase**. Este enfoque facilita la **Integración Continua (CI)**, permitiendo desplegar y validar las Reglas de Seguridad de Firestore y la lógica de autenticación en un 'núcleo' estable, antes de iterar sobre módulos más complejos. De este modo, aseguramos que cada incremento sea compatible con la infraestructura en la nube desde el primer día, minimizando errores de latencia o permisos en etapas avanzadas del desarrollo.

2.2. Diseño de la Aplicación y sus Fases

Para la ejecución de este ciclo, el proyecto se divide en las siguientes etapas integradas:

- **Fase de Análisis (Estática):** Definición del esquema NoSQL en Firestore. Es crítica en el modelo incremental porque un error en el diseño de la colección Usuarios afectaría a todos los incrementos posteriores.
- **Fase de Implementación del Núcleo (Seguridad):** Implementación de Firebase Auth y reglas de seguridad. Este es el **"Incremento Crítico"**, donde se valida que un cliente no pueda acceder a la recaudación de la peluquería.
- **Fases de Desarrollo Funcional (Sprints):** Cada incremento añade una capa de valor (primero gestión de empleados, luego motor de citas, finalmente notificaciones).

Ventajas	Inconvenientes
Mitigación de riesgos técnicos: Al integrar Firebase desde el primer incremento, detectamos problemas de latencia o conectividad meses antes de la entrega final.	Rigidez de la Base: Requiere una definición muy precisa de los objetos Java (POJOs). Si en la semana 8 decidimos cambiar cómo se guarda una cita, el impacto es alto.
Entregas funcionales tempranas: El dueño de la peluquería puede empezar a usar el panel de gestión para organizar su	Gestión de dependencias: El módulo de notificaciones (FCM) no puede probarse hasta que el motor de reservas esté 100%

Nota Técnica sobre Esfuerzo y Recursos:

Para garantizar la coherencia con el Estudio de Viabilidad , la planificación del diagrama de Gantt se basa en un esfuerzo total de **375 horas/persona**.

- **Distribución de Carga:** Durante la **Fase 4 (Semanas 6 a 9)**, que representa el núcleo del desarrollo (220h), el cronograma refleja el trabajo intensivo del *Desarrollador Full Stack* (40h/semana) apoyado por sesiones de validación y ajustes de diseño por parte del *Diseñador UX* y el *Tester QA*.
- **Hitos de Entrega:** Cada barra del diagrama representa tareas específicas del **Ciclo de Vida Incremental**. El solapamiento parcial en la Semana 9 entre "Desarrollo" y "QA" justifica la transición fluida hacia el cierre del proyecto, asegurando que el presupuesto operativo se mantenga dentro de los márgenes previstos.
- **Recursos Materiales:** Todas las tareas asignadas en este cronograma cuentan con la infraestructura descrita en el apartado de viabilidad (Android Studio, Firebase Spark Plan y estaciones de trabajo con 16GB RAM).